

基于地理圈层的可解释房产决策支持系统 (GC-REDSS)

概述

GC-REDSS 是一个基于大数据的房产选择决策支持系统，解决了传统搜索筛选机制的僵化限制。系统不使用硬性阈值，而是采用：

- **静态等时线分析（地理圈层）**：基于 OpenStreetMap 数据计算，定义灵活的"通勤"和"生活"可达性区域
- **弹性过滤**：透明的加权评分模型，提供可调节的 S 分数，允许用户评估即使略超出定义圈层的房产
- **多交通模式分析**：支持驾车、步行和公共交通模式

功能特性

- **数据处理**：基于 PySpark 的数据清洗和特征工程
- **地理空间分析**：使用 OSMnx 和 NetworkX 进行等时线计算
- **弹性评分模型**：可配置权重的加权评分系统
- **多交通模式支持**：计算不同交通方式的通勤圈
- **Docker 支持**：使用 Docker 和 Docker Compose 轻松部署
- **REST API**：可选的基于 Flask 的 API，用于 Web 集成

项目结构

.	
├─ src/	
└─ __init__.py	
└─ config.py	# 配置设置
└─ data_processor.py	# PySpark 数据处理
└─ geo_circle.py	# 等时线计算
└─ scoring_model.py	# 弹性过滤评分
└─ main.py	# 主程序入口
└─ api.py	# 可选的 REST API (Flask)
└─ utils.py	# 工具函数
├─ examples/	
└─ example_usage.py	# 使用示例脚本
├─ data/	
└─ raw/	# 原始 CSV 数据
└─ processed/	# 处理后的 Parquet 文件
├─ output/	# 输出文件（评分后的房产）
├─ requirements.txt	# Python 依赖
├─ Dockerfile	# Docker 配置
├─ docker-compose.yml	# Docker Compose 配置
├─ README.md	# 英文文档
└─ README_CN.md	# 本文档（中文）

系统工作流程

GC-REDSS 系统遵循三阶段处理流程：

输入数据

CSV 文件: **NY-House-Dataset.csv**

- 房产地址、价格、卧室数、浴室数等
- 经纬度坐标



阶段 1: 数据处理

DataProcessor 模块

- └ 加载原始 CSV 数据
- └ 数据清洗 (去重、验证坐标)
- └ 特征工程 (价格/平方英尺、卧室/浴室比例等)
- └ 输出: 清洗后的 **Spark DataFrame** + **Parquet** 文件



阶段 2: 地理圈层计算

GeoCircleCalculator 模块

- └ 地理编码工作地址 → 坐标
- └ 从 **OpenStreetMap** 下载路网
- └ 计算通勤等时线 (驾车/步行/公交)
- └ 计算生活便利性等时线 (步行)
- └ 输出: 每个圈层的多边形对象



阶段 3: 评分与排序

ScoringModel 模块

- └ 计算各维度分数
 - 价格分数 (归一化, 越低越好)
 - 通勤分数 (基于地理圈层)
 - 生活便利性分数
 - 房产面积、卧室数、浴室数分数
- └ 应用弹性过滤 (圈外指数衰减)
- └ 加权组合 → **S** 分数
- └ 输出: 评分后的 **DataFrame** + **Top N CSV**



输出结果

- **scored_properties.parquet** (完整数据集及分数)

- top_properties.csv (Top N 房产)
- 控制台输出 (排名列表)

详细工作流程步骤

1. 数据加载与清洗

- 将 CSV 文件加载到 Spark DataFrame
- 删除重复记录
- 验证和清洗坐标 (纬度: -90 到 90, 经度: -180 到 180)
- 过滤缺少关键数据的记录 (价格、坐标)
- 标准化地址字段

2. 特征工程

- 计算 PRICE_PER_SQFT (价格 / 平方英尺)
- 计算 BEDS_PER_BATH 比例
- 从行政区域提取 BOROUGH
- 标准化房产类型

3. 地理圈层计算

- 使用 Geopy 将工作地址转换为坐标
- 从 OpenStreetMap 下载路网 (缓存以提高性能)
- 计算从工作地点到所有可达节点的最短路径时间
- 为每种交通模式生成等时线多边形
- 创建生活便利性圈 (步行距离到便利设施)

4. 评分与排序

- 对每个房产:
 - 检查是否在每个地理圈层内/外
 - 如果在圈外, 计算到圈边界的距离
 - 应用弹性过滤 (指数衰减: $\exp(-\text{penalty} * 2)$)
 - 将所有特征归一化到 [0, 1] 范围
 - 计算加权 S 分数
- 按 S 分数排序并返回 Top N

安装

使用 Docker (推荐)

1. 构建并启动容器:

```
docker-compose up --build
```

2. 访问 JupyterLab:

- 在浏览器中打开 `http://localhost:8888`
- 项目文件将在工作区中可用

3. 停止容器:

```
docker-compose down
```

本地安装

1. 安装依赖:

```
pip install -r requirements.txt
```

2. 确保已安装 Java (PySpark 需要) :

- Java 8 或更高版本
- 验证: `java -version`

3. 设置数据目录:

- 将 CSV 文件放置在 `data/raw/NY-House-Dataset.csv`

使用方法

方法 1: 命令行界面 (CLI)

基本用法:

```
python src/main.py \  
  --work-address "Times Square, New York, NY" \  
  --commute-threshold 30 \  
  --life-threshold 15 \  
  --top-n 20
```

自定义输入/输出:

```
python src/main.py \  
  --work-address "Central Park, New York, NY" \  
  --commute-threshold 45 \  
  --input-file "data/raw/custom_data.csv" \  
  --output-file "output/my_results.parquet" \  
  --top-n 50
```

跳过数据处理 (使用已有处理数据) :

```
python src/main.py \
  --work-address "Brooklyn Bridge, New York, NY" \
  --skip-processing \
  --commute-threshold 30
```

在 Docker 容器中：

```
# 首先设置 PYTHONPATH
cd /home/jovyan/work
export PYTHONPATH=/home/jovyan/work:$PYTHONPATH

# 或使用模块语法
python -m src.main \
  --work-address "Times Square, New York, NY" \
  --commute-threshold 30 \
  --life-threshold 15 \
  --top-n 20
```

命令行参数：

参数	类型	必需	默认值	说明
--work-address	str	✓	-	工作地点地址
--commute-threshold	float	✗	30	最大通勤时间（分钟）
--life-threshold	float	✗	15	到便利设施的最大步行时间（分钟）
--input-file	str	✗	data/raw/NY-House-Dataset.csv	输入 CSV 文件路径
--output-file	str	✗	output/scored_properties.parquet	输出文件路径
--skip-processing	flag	✗	False	跳过数据处理，使用已有数据
--top-n	int	✗	20	显示前 N 个房产

方法 2: Python API（编程方式）

基本用法：

```
from pyspark.sql import SparkSession
from src.data_processor import DataProcessor
from src.scoring_model import ScoringModel

# 初始化 Spark
spark = SparkSession.builder.appName("GC-REDSS").getOrCreate()

# 步骤 1: 处理数据
processor = DataProcessor(spark)
properties_df = processor.process()

# 步骤 2: 评分房产
scoring_model = ScoringModel()
scored_df = scoring_model.score_properties(
    properties_df,
    work_address="Times Square, New York, NY",
    commute_threshold_minutes=30,
    life_threshold_minutes=15
)

# 步骤 3: 获取 Top 房产
from pyspark.sql.functions import col
top_10 = scored_df.orderBy(col("SCORE_S_SCORE").desc()).limit(10)
top_10.show()
```

使用自定义权重:

```

from src.scoring_model import ScoringModel

# 定义自定义权重
custom_weights = {
    "price": 0.4,          # 更重视价格
    "commute_time": 0.3,   # 更重视通勤
    "life_accessibility": 0.1,
    "property_size": 0.1,
    "bedrooms": 0.05,
    "bathrooms": 0.05,
}

# 使用自定义权重
scoring_model = ScoringModel(weights=custom_weights)
scored_df = scoring_model.score_properties(
    properties_df,
    work_address="Times Square, New York, NY",
    commute_threshold_minutes=30
)

```

方法 3: Jupyter Notebook

1. 启动 Docker 容器:

```
docker-compose up
```

2. 打开 JupyterLab: <http://localhost:8888>

3. 创建新 notebook 并运行:

```

import sys
from pathlib import Path
sys.path.insert(0, str(Path.cwd()))

from pyspark.sql import SparkSession
from src.data_processor import DataProcessor
from src.scoring_model import ScoringModel

# 你的代码...

```

方法 4: REST API

启动 API 服务器:

```
python src/api.py
```

API 将在 `http://localhost:5000` 可用

API 端点:

1. 健康检查

GET `http://localhost:5000/health`

响应:

```
{
  "status": "healthy",
  "service": "GC-REDSS"
}
```

2. 评分房产

POST `http://localhost:5000/api/score`

Content-Type: `application/json`

```
{
  "work_address": "Times Square, New York, NY",
  "commute_threshold": 30,
  "life_threshold": 15,
  "top_n": 20,
  "weights": {
    "price": 0.3,
    "commute_time": 0.25,
    "life_accessibility": 0.15,
    "property_size": 0.1,
    "bedrooms": 0.1,
    "bathrooms": 0.1
  }
}
```

响应:

```
{
  "status": "success",
  "count": 20,
  "properties": [
    {
      "ADDRESS": "...",
      "PRICE": 315000,
      "SCORE_S_SCORE": 0.85,
      ...
    },
    ...
  ]
}
```

3. 获取配置

GET http://localhost:5000/api/config

4. 更新权重

POST http://localhost:5000/api/config/weights
Content-Type: application/json

```
{
  "weights": {
    "price": 0.4,
    "commute_time": 0.3,
    ...
  }
}
```

使用 curl:

```
# 评分房产
curl -X POST http://localhost:5000/api/score \
  -H "Content-Type: application/json" \
  -d '{
    "work_address": "Times Square, New York, NY",
    "commute_threshold": 30,
    "top_n": 10
  }'
```

```
# 健康检查
curl http://localhost:5000/health
```

使用 Python requests:

```
import requests

# 评分房产
response = requests.post(
    'http://localhost:5000/api/score',
    json={
        'work_address': 'Times Square, New York, NY',
        'commute_threshold': 30,
        'life_threshold': 15,
        'top_n': 20
    }
)

results = response.json()
print(f"找到 {results['count']} 个房产")
for prop in results['properties']:
    print(f"{prop['ADDRESS']}: 分数 {prop['SCORE_S_SCORE']:.3f}")
```

在 Docker 中运行 API:

```
# 在 docker-compose.yml 中添加 API 服务，或在容器中运行
docker exec -it gc-redss-container python src/api.py
```

配置

编辑 src/config.py 进行自定义:

评分权重

```
DEFAULT_SCORING_WEIGHTS = {
    "price": 0.3,
    "commute_time": 0.25,
    "life_accessibility": 0.15,
    "property_size": 0.1,
    "bedrooms": 0.1,
    "bathrooms": 0.1,
}
```

Spark 配置

```
SPARK_CONFIG = {  
    "spark.app.name": "GC-REDSS",  
    "spark.master": "local[*]",  
    "spark.driver.memory": "4g",  
    "spark.executor.memory": "4g",  
}
```

地理圈层设置

```
DEFAULT_COMMUTE_THRESHOLD = 30 # 分钟  
DEFAULT_LIFE_THRESHOLD = 15 # 分钟  
DEFAULT_TRANSPORT_MODES = ["driving", "walking", "transit"]
```

OSMnx 设置

```
OSMNX_SETTINGS = {  
    "network_type": "all",  
    "timeout": 180,  
    "max_query_area_size": 50 * 1000 * 50 * 1000,  
}
```

评分模型

评分模型使用弹性过滤，包含以下组件：

1. **价格分数**（权重 0.30）：归一化价格（越低越好）
2. **通勤分数**（权重 0.25）：基于到通勤圈的距离
 - 圈内：分数 = 1.0
 - 圈外：分数 = $\exp(-\text{penalty_factor} * 2)$
 - 支持多种交通模式（驾车、步行、公交）
3. **生活便利性分数**（权重 0.15）：到便利设施的步行距离
4. **房产面积分数**（权重 0.10）：归一化平方英尺
5. **卧室分数**（权重 0.10）：归一化卧室数
6. **浴室分数**（权重 0.10）：归一化浴室数

最终 S 分数：

$S_score = \sum(\text{特征分数} \times \text{权重})$

弹性过滤公式：

如果房产在圈内：

分数 = 1.0

否则：

到边界距离 = 计算距离(房产, 圈)

时间惩罚 = 到边界距离 / 平均速度

惩罚因子 = 时间惩罚 / 阈值时间

分数 = $\exp(-\text{惩罚因子} * 2)$

输出

系统生成：

1. **处理后的数据**： data/processed/processed_houses.parquet
 - 清洗和特征工程后的数据
 - Parquet 格式，存储高效
2. **评分后的房产**： output/scored_properties.parquet
 - 所有房产及完整评分信息
 - 包含所有原始字段 + 衍生特征 + 所有分数
3. **Top 房产 CSV**： output/top_properties.csv
 - 人类可读的 CSV 格式
 - 按 S 分数排序的 Top N 房产
 - 易于导入 Excel 或其他工具

输出列：

- 原始： ADDRESS , PRICE , BEDS , BATH , PROPERTYSQFT , LATITUDE , LONGITUDE 等
- 衍生： PRICE_PER_SQFT , BEDS_PER_BATH , BOROUGH 等
- 分数： SCORE_PRICE_SCORE , SCORE_COMMUTE_SCORE , SCORE_LIFE_SCORE , SCORE_S_SCORE 等

技术栈

- **PySpark**：大数据处理和分布式计算
- **OSMnx**：OpenStreetMap 网络分析和路网提取
- **NetworkX**：图论，用于最短路径和路线计算
- **Shapely**：几何操作和多边形计算
- **Geopy**：地理编码（地址转坐标）
- **Pandas/NumPy**：数据操作和数值计算
- **Flask**：REST API 框架（可选）
- **Plotly**：可视化（未来使用）

故障排除

ModuleNotFoundError: No module named 'src'

解决方案 1: 设置 PYTHONPATH

```
export PYTHONPATH=/path/to/project:$PYTHONPATH
python src/main.py ...
```

解决方案 2: 使用模块语法

```
python -m src.main ...
```

解决方案 3: 从项目根目录运行

```
cd /path/to/project
python src/main.py ...
```

OSMnx 网络下载缓慢

- 首次运行会下载路网数据（可能需要几分钟）
- 网络数据会被缓存，后续运行更快
- 考虑在配置中减小 `max_query_area_size` 以加快下载

Spark 内存错误

- 在 `src/config.py` 中增加内存：

```
"spark.driver.memory": "8g",
"spark.executor.memory": "8g",
```

API 无法启动

- 确保 Flask 已安装： `pip install flask flask-cors`
- 检查端口 5000 是否被占用
- 使用调试模式运行： `python src/api.py`

未来增强

- ☐ SHAP/LIME 集成，用于模型可解释性
- ☐ 使用 Plotly 的交互式可视化

- ☐ Streamlit/Dash Web 界面
- ☐ 实时房产更新
- ☐ 高级聚类分析
- ☐ 价格预测的机器学习模型
- ☐ 与房产 API 集成 (Zillow、Redfin)

许可证

本项目仅用于教育目的。

参考资料

- 数据集: [New York Housing Market](#)
- OSMnx: [文档](#)
- PySpark: [文档](#)